Chris Toomey, Jenice An, and Tongyu Guo
ADAN 8888 - Applied Analytics Project - Spring 2026
Module 4 Assignment - Make Data Model Ready

HelpHerInvest: A machine learning-based investment recommendation system that aligns user interests and risk preferences with benchmark stock evaluation.

Semantic Natural Language Processing

Data Preprocessing

Dealing with Null Values / Missing Data:

The dataset underwent a combination of record-level cleaning and text-focused preprocessing to prepare it for downstream NLP modeling. First, the data was filtered to ensure each observation could be reliably identified: rows missing a valid stock identifier (symbol) were removed. In addition, several common placeholder strings that often appear in scraped or exported datasets (such as empty strings, "None", "NaN", and "[]") were systematically converted into true missing values (NaN). This step standardizes missingness across the dataset and prevents placeholder artifacts from being treated as meaningful text or categories during modeling.

Normalize / Standardize the Data:

A key part of the cleaning process involved transforming the semi-structured company officers field into a usable format. In the raw data, officer information was stored as a string that represents a list of dictionaries. This column was parsed safely into a structured list, with error handling to avoid failures when malformed values were encountered. The structured officer records were then normalized into a single human-readable text string, ensuring that relevant executive information could be incorporated into the company's overall text representation in a consistent way.

After standardizing individual fields, the pipeline constructed a single consolidated document per company by concatenating descriptive attributes such as company name, sector, industry, long business summary, and the cleaned officer text. Empty or whitespace-only values were excluded during concatenation, which reduces noise and prevents the creation of documents dominated by missing fields. Rows that still produced no usable text after this merge step were

removed, ensuring that every retained record contains sufficient textual information for NLP feature extraction.

Textual Data Preprocessing:

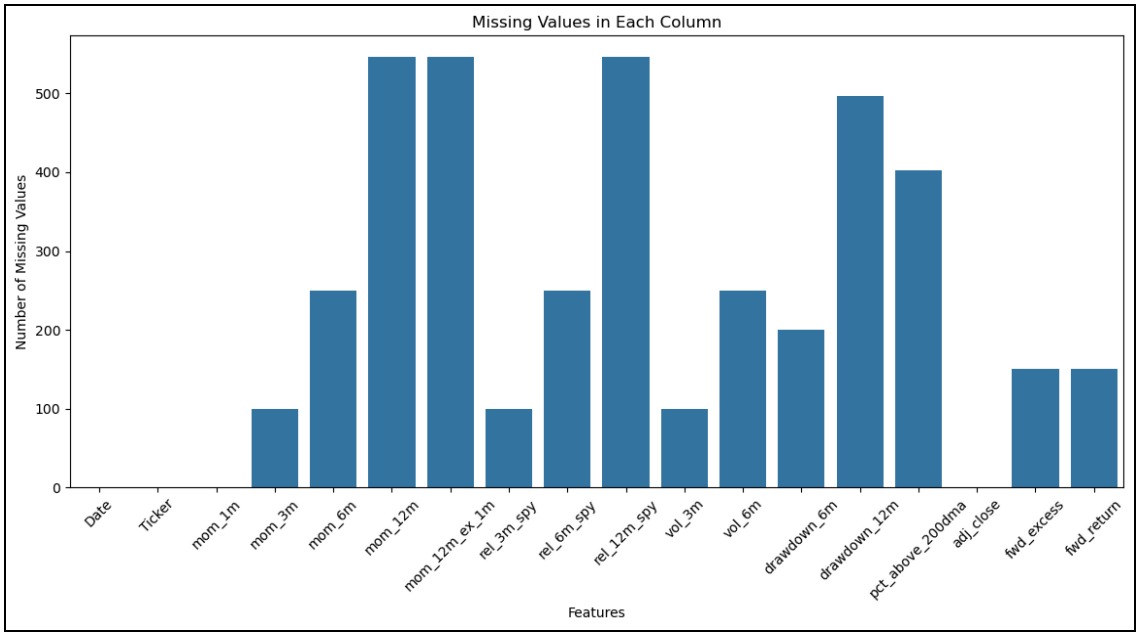
Finally, the text was cleaned specifically for TF-IDF style modeling. This included Unicode normalization, conversion to lowercase, and removal of HTML fragments, URLs, and email addresses, which are generally not informative for company description similarity or classification tasks. Additional normalization removed non-alphabetic characters, collapsed repeated whitespace, and optionally removed English stopwords and very short tokens. The result is a clean, standardized text field (document_clean_tfidf) that is better suited for vectorization, reduces vocabulary sparsity, and improves the quality and stability of text-based models.

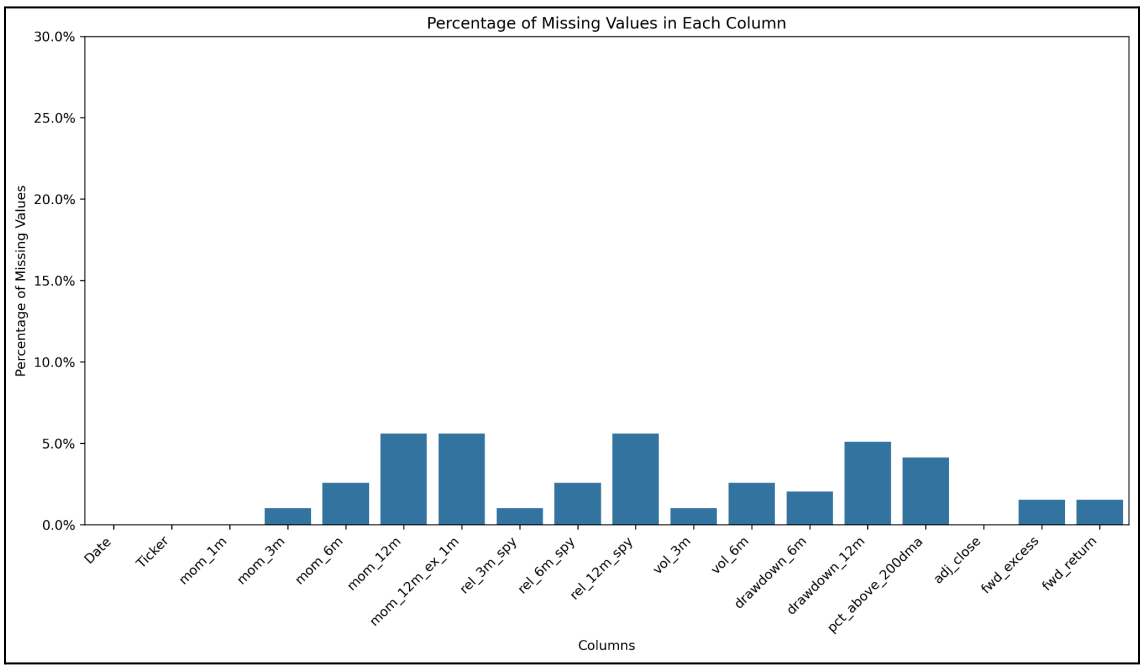
Multinomial Classification Model

Data Preprocessing

Dealing with Null Values:

The time-series financial dataset we are working with contains null values across 14 of our variables, including the predictor variable forward excess return ("fwd_excess"). This is expected and these values are not missing at random because they are undefined until enough historical data exists to produce the value. For example, when calculating the 12-month momentum of a stock ticker, the formula requires 12 periods in order to calculate the momentum. This rolling period results in instances where the beginning periods of the dataset (i.e. first 12 months for each ticker) will naturally contain null values. The same goes for the forward excess return - this is calculated by comparing the stock ticker's price today to the price in 3 months. This results in the tail end of the dataset containing null values for the forward excess return because the predictions can not look ahead in the future to calculate. The number of null values per each column seems high on the first barplot, but when looking at the percentage of the dataset, it isn't as impactful (ranging from 1% - 5.6%). Nonetheless, this creates a challenge for our project and how we deal with these null values.





There are different ways to handle null values within a dataset, such as imputation (mean, median, etc.), forward-fill / backward-fill, replace with zeros, or dropping the rows all together. Surprisingly, many resources recommend dropping the rows all together, which is not typically best practice when dealing with null values in a dataset. The reason for this is that these values

are not missing at random because they rely on historical data to be produced. Performing imputation (such as by mean or median) or forward-fill / backward-fill would create artificial values and could distort the statistical properties of the rolling variables. This could have a negative influence on the model and skew the results. One method to alleviate dropping rows that contain rolling statistics with null values is to expand the timeframe (i.e. adding one year) of the dataset we are creating. By expanding the start date of the dataset range by one year, this can maintain the same number of rows we had previously developed.

Duplicate Rows and/or Columns:

We performed a check to see if there were any duplicate rows and/or duplicate columns and there weren't any. This validated that the creation of our dataset did not mistakenly extract the same symbols or ETFs when pulling the data from the SEC.

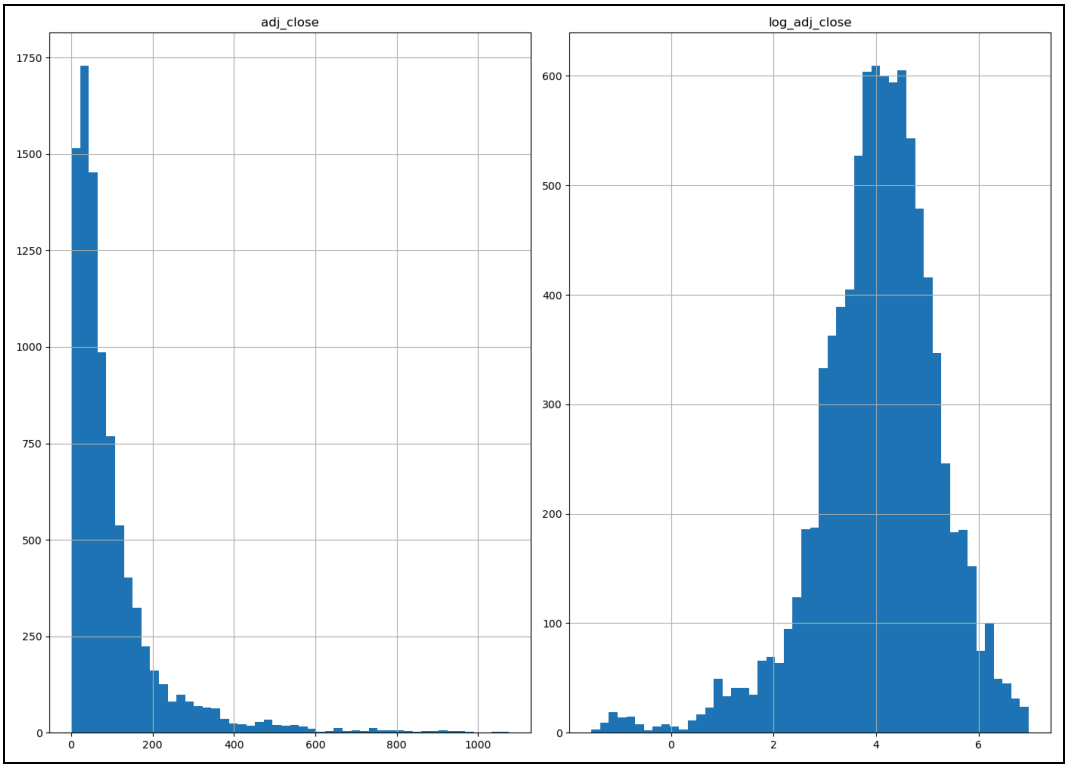
```
# check if there are duplicate rows
num_dupes = df.duplicated().sum()

if num_dupes > 0:
    print("Duplicate rows found. Consider removing them.")
    # View duplicate rows
    dupe_rows = df[df.duplicated()]
    print(dupe_rows)
else:
    print("No duplicate rows found.")

# check if there are duplicate columns
dup_content_mask = df.T.duplicated().sum()

if dup_content_mask > 0:
    print("Duplicate columns found. Consider removing them.")
    # View duplicate columns
    dupe_cols = df.columns[df.T.duplicated()]
    print(dupe_cols)
else:
    print("No duplicate columns found.")
```

✓ 0.4s

```
No duplicate rows found.
No duplicate columns found.
```

Data Transformations:

Many of the variables being used are already in a standardized format since they are mostly represented as percentages. However, the adjusted close price has a wide range of values, displaying a maximum value of 1075.47, a minimum value of 0.21, and a mean value of 100.67. Since this range displayed a significant skew, we decided to create a new logarithmic value of each adjusted close price to make the distribution more normal. This transformation appears to be successful based on the results in the histogram below.


adj_close
1750
1500
1250
1000
750
500
250
0
0
200
400
600
800
1000
log_adj_close
600
500
400
300
200
100
0
0
2
4
6